

Data Mining Practice7 문벼리

Task 1: Movie Analysis

영화들의 평균 평점과 평점의 수를 비교해보자. 평점의 수가 높은 영화는 많은 관객들이 감상한 인기있는 영화라고 생각할 수 있다. 인기 있는 영화일수록 평균 평점이 높다고 볼 수 있는가?

```
load("recommender_watcha.RData")
#movie_train 데이터에서 영화별로 평균 평점과 평점을 남긴 수를 계산
movie_rating_data <- movie_train %>%
  group_by(movie_title) %>%
  summarise(average_rating = mean(score, na.rm = TRUE),
            number_of_ratings = n()) %>%
  ungroup()

#평균 평점을 기준으로 내림차순
movie_rating_data %>%
  arrange(-average_rating)

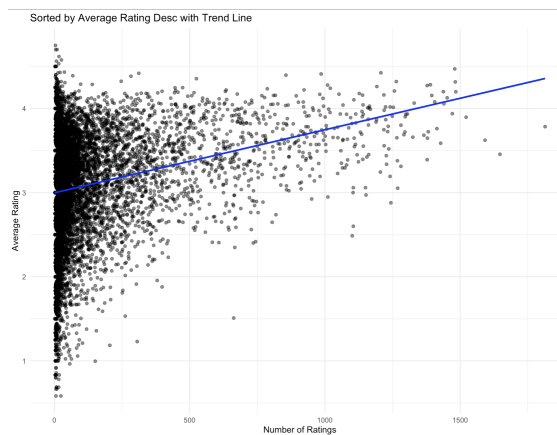
#평점 수를 기준으로 내림차순
movie_rating_data %>%
  arrange(-number_of_ratings)

# 1. 추세선 추가한 그래프
plot1 <- movie_rating_data %>%
  ggplot(aes(x = number_of_ratings, y = average_rating)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE, color = "blue") + #
```

```
labs(title = "Sorted by Average Rating Desc with Trend Line",
      x = "Number of Ratings",
      y = "Average Rating") +
theme_minimal()
```

plot1

Result



추세선을 통해 확인해봤을 때, rating 수가 높으면 평균 평점이 높은 경향성이 있어보인다. 이를 통계적으로 판단하기 위해 cor.test를 진행했습니다.

```
# 상관관계 계산
cor.test(movie_rating_data$number_of_ratings, movie_rating_da
```

Pearson's product-moment correlation

```
data: movie_rating_data$number_of_ratings and
movie_rating_data$average_rating
t = 25.83, df = 11014, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.2213062 0.2565223
sample estimates:
      cor
0.2389928
```

H0(귀무가설): 평점 수 (number_of_ratings)와 평균 평점 (average_rating) 간의 상관관계가 0이다.

H1(대립가설): 평점 수 (number_of_ratings)와 평균 평점 (average_rating) 간의 상관관계가 0이 아니다.

→ 상관계수는 약 0.238로 영화의 평점을 남긴 횟수와 평균 평점 사이에 약한 양의 상관계수를 가집니다.

p-value < 2.2e-16 값으로 매우 작아서 통계적으로 유의미하다고 볼 수 있습니다. (귀무가설 기각)

즉, 약한 양의 상관관계가 있으며, 이 상관계수는 통계적으로 유의미합니다.

숨겨진 명작 찾기

```
# 평점 수와 평균 평점의 25% 기준 값 계산
lower_bound_ratings <- quantile(movie_rating_data$number_of_ratings, 0.25)
upper_bound_avg_rating <- quantile(movie_rating_data$average_rating, 0.25)

# 평점 수가 적고 평균 평점이 높은 영화 필터링
hidden_gems <- movie_rating_data %>%
  filter(number_of_ratings <= lower_bound_ratings & average_rating >= upper_bound_avg_rating)
  arrange(-average_rating)

# 숨겨진 명작 출력
print(hidden_gems)

##명작이라고 하기엔 너무 낮은 사람의 평점수라고 판단함.
lower_bound_ratings_50 <- max(quantile(movie_rating_data$number_of_ratings, 0.25), 50)
upper_bound_avg_rating_50 <- quantile(movie_rating_data$average_rating, 0.25)

# 평점 수가 50 이상이고 평균 평점이 높은 영화 필터링
hidden_gems_over50 <- movie_rating_data %>%
  filter(number_of_ratings >= 50 & number_of_ratings <= lower_bound_ratings_50 & average_rating >= upper_bound_avg_rating_50)
  arrange(-average_rating)
```

```
# 숨겨진 명작 출력
print(hidden_gems_over50)
```

result

평점 수 하위 25%와 평점 평균 상위 25% 결과

```
# A tibble: 632 x 3
  movie_title      average_rating number_of_ratings
  <chr>          <dbl>          <int>
1 그 노래를 기억하세요? 4.75             4
2 시테라 섬으로의 여행 4.7              5
3 나의 계곡은 푸르렀다 4.56             8
4 베크마이스터 하모니즈 4.5              7
5 빛을 향한 그리움 4.5              3
6 소서러 4.5              5
7 음베르토 D 4.5              2
8 인간의 조건 4.5              5
9 공포의 보수 4.4              5
10 바르피! 4.4              5
```

명작이라고 하기엔 너무 낮은 사람의 평점수라고 판단하여 최소 50명이상이 평가를 해야한다고 설정

평점 수 하위 25%인데 50명 이상 평가를 받은 작품과 평점 평균 상위 25%의 결과

```
# A tibble: 9 x 3
  movie_title      average_rating number_of_ratings
  <chr>          <dbl>          <int>
1 선셋 대로 4.13             50
2 체리향기 3.94             50
3 내 친구의 집은 어디인가 3.92             50
4 카지노 3.87             50
5 말하는 건축가 3.72             50
6 아메리칸 허니: 방황하는 별의 노래 3.72             50
7 트랜스 3.57             50
8 굿모닝 맨하탄 3.5             50
9 로미오: 눈을 가진 죄 3.5             50
```

카지노를 본 사람들의 말을 들어본 결과, 최민식 배우의 연기가 최고였지만 결말이 아쉬웠다고 들었습니다. 카지노 정도면 명작으로 볼 수 있을 것 같습니다.

호불호가 강하거나 피해야 하는 작품

```
# 평점 수와 평균 평점의 25% 기준 값 계산
upper_bound_ratings <- quantile(movie_rating_data$number_of_r
lower_bound_avg_rating <- quantile(movie_rating_data$average_
```

```
# 평점 수가 많고 평균 평점이 낮은 영화 필터링
controversial_or_avoid_movies <- movie_rating_data %>%
  filter(number_of_ratings >= upper_bound_ratings & average_r

# 호불호가 강하거나 피해야 할 작품 출력
print(controversial_or_avoid_movies)
```

Result

```
# A tibble: 440 × 3
  movie_title          average_rating number_of_ratings
  <chr>              <dbl>             <int>
1 #살아있다          2.13               305
2 007 어나더데이     2.69               107
3 10억               2.20               371
4 13 고스트          2.69               151
5 1번가의 기적       2.50               248
6 2009 로스트 메모리즈 2.53               155
7 2016 무한상사 - 위기의 회사원 2.49               261
8 365일              1.06                86
9 4교시 추리영역     1.67                83
10 50가지 그림자: 심연 2.25               105
# i 430 more rows
# i Use `print(n = ...)` to see more rows
```

개인적으로 유아인 배우가 나오는 '살아있다' 작품을 시청했는데 나름 재밌었는데 평점이 생각보다 너무 낮아 놀랐다. 무한상사 또한 재밌는데 생각보다 평점이 낮다는 게 신기했다.

Task 2: Book Analysis

```
book_train <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학
기/데이터 마이닝 실습/Data-Mining-Practicum/myR/dataset for rec
ommendation watcha/book_train.csv",
  fileEncoding = "CP949", # 또는 "EUC-K
R"
  stringsAsFactors = FALSE)
```

```
book_tast <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/dataset for recommendation watcha/book_test.csv",
                     fileEncoding = "CP949", # 또는 "EUC-K
R"
                     stringsAsFactors = FALSE)
```

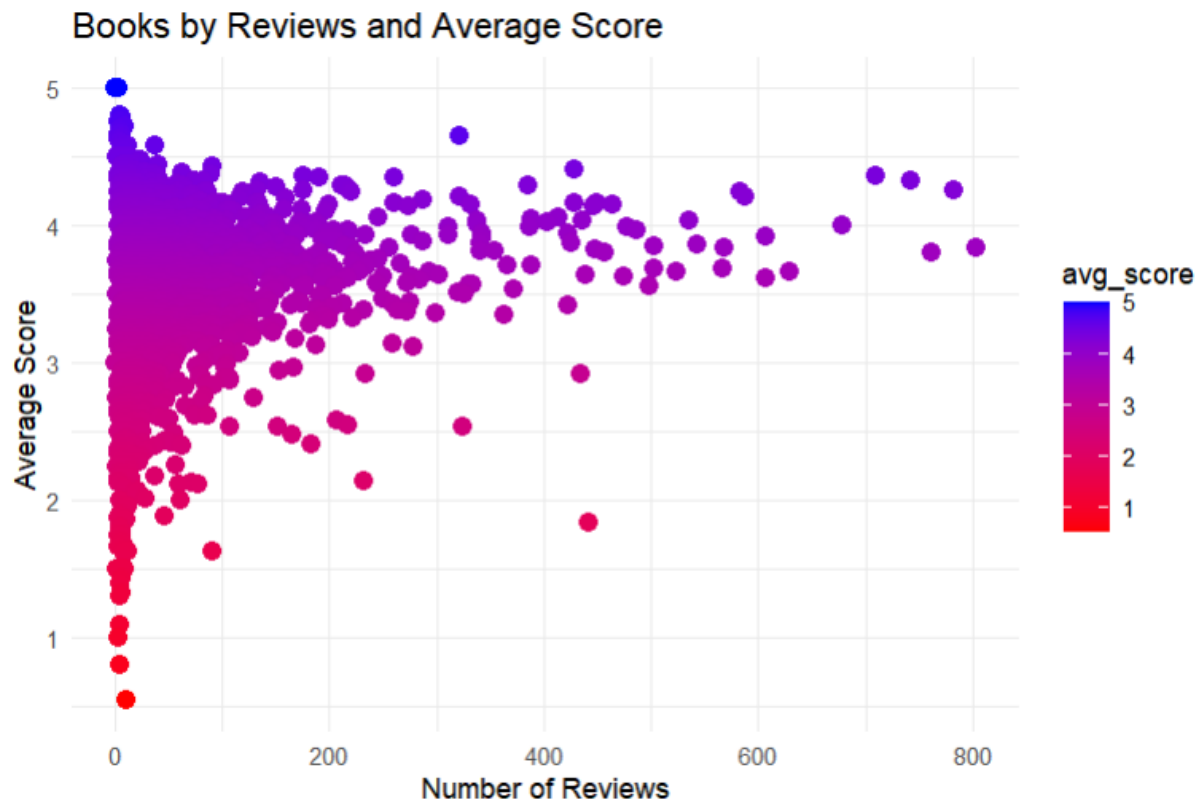
```
book_train<-book_train %>%
  group_by(book_code)%>%
  mutate(reviews = n())

head(book_train)
book_train %>%
  group_by(book_code) %>%
  mutate(avg_score = mean(score))%>%
  arrange(desc(avg_score))
```

평균 평점과, 각 book별 리뷰의 갯수를 세어 저장함

한번 전체 book에 대해서 평균 점수와 리뷰 수를 시각화해 보았다

```
ggplot(book_train, aes(x = reviews, y = avg_score, label =
book_title)) +
  geom_point(aes(color = avg_score), size = 3) + # 각 책을 점
으로 표시, 평균 평점에 따라 색상 지정
  scale_color_gradient(low = "red", high = "blue") + # 색상
변화 (평균 점수에 따라)
  labs(title = "Books by Reviews and Average Score",
        x = "Number of Reviews",
        y = "Average Score") +
  theme_minimal()
```



상당수의 책들이 리뷰 수가 200개 안쪽에서 많이 이루어지고 있으며, 리뷰 수가 10개가 채 되지 않는 도서가 상당한 것 같다.

평균 평점이 높은 Book 상위 10개

```
book_train %>%
  arrange(desc(avg_score))
  user_code book_code score book_title
reviews avg_score
  <chr>      <chr>      <dbl> <chr>
<int>      <dbl>
1 US000274 BC0011134 5 세월호, 그날의 기록
4 5
2 US000950 BC0018957 5 웅크린 말들
1 5
3 US000183 BC0008295 5 매잡이
5 4.8
```

4	US000307	BC0012317	5	일방통행로 / 사유이미지
7				4.79
5	US000086	BC0006213	5	A Game of Thrones
4				4.75
6	US001084	BC0023719	5	소설 영웅문 제1부 - 몽고의 별
6				4.75
7	US000159	BC0007898	5	눈물도 빛을 만나면 반짝인다
9				4.72
8	US000130	BC0007430	4	여자는 인질이다
7				4.71
9	US001327	BC0027795	4.5	소설 영웅문 제2부 - 영웅의 별
5				4.7
10	US000291	BC0001481	5	사자왕 형제의 모험
3				4.67

review수가 10개가 넘지 못하는 book들이 가장 상단에 위치하여 있다.

이번에는 review의 수를 50개 이상으로 확인해 보고자 한다

```
book_train %>%
  filter(reviews > 50)%>%
  arrange(desc(avg_score))
  user_code book_code score book_title          review
s avg_score
  <chr>      <chr>      <dbl> <chr>          <int>
>      <dbl>
1 US000007   BC0001328   3.5 슬램덩크      322
4.66
2 US000014   BC0002526   4 시선으로부터,    91
4.43
3 US000010   BC0001577   5 소년이 온다      429
4.40
4 US000014   BC0002748   5 정확한 사랑의 실험 63
4.39
5 US000018   BC0002244   4 H2                8
9 4.38
6 US000003   BC0000146   5 강철의 연금술사   176
4.36
```


7	US000003	BC0000359	5	해리 포터와 마법사의 돌	710
4.36					
8	US000003	BC0000117	4	여중생A	261
4.35					
9	US000008	BC0001044	5	입 속의 검은 잎	191
4.35					
10	US000006	BC0001106	4.5	존재의 세 가지 거짓말	82
4.33					

리뷰 수를 50개 이상으로 하니 다양한 책이 많이 나오는 것 같다. 눈에 띄는 것은 소년이 온다, 해리포터, 슬램덩크 등 많은 사람들이 읽은 인기있는 도서들의 평점이 위로 올라온 것이 보인다.

가장 많은 리뷰가 있는 Book 10개

user_code	book_code	score	book_title	review
s avg_score				
<chr>	<chr>	<dbl>	<chr>	<int>
>	<dbl>			
1	US000007	BC0000691	4.5 82년생 김지영	803
3.84				
2	US000001	BC0000051	4.5 어린 왕자	782
4.25				
3	US000007	BC0001348	3 나미야 잡화점의 기적	761
3.81				
4	US000003	BC0000351	4 1984	74
2	4.32			
5	US000003	BC0000359	5 해리 포터와 마법사의 돌	710
4.36				
6	US000001	BC0000063	5 데미안	678
4.00				
7	US000003	BC0000314	3.5 채식주의자	630
3.66				
8	US000006	BC0001186	2 노르웨이의 숲	607
3.92				
9	US000003	BC0000346	4 연금술사	607

3.62

10 US000003 BC0000358 4 해리 포터와 비밀의 방 587

4.21

이번에는 많은 사람이 읽은 리뷰가 많은 순서대로 정렬하였다. 각각에 대해 평점을 확인해보면 3점대~4점대 사이의 평점을 받고 있음을 알 수 있다.

평점이 높지만 리뷰가 적은 Book 10개

리뷰가 10개 미만인 book에서 평점별로 정렬을 한다

```
top_books <- book_train %>%
  filter(reviews < 10) %>% # 리뷰 수가 적은 책 필터링 (예: 리뷰
  가 10 미만인 책)
  arrange(desc(avg_score)) %>% # 평균 평점이 높은 순서로 정렬
  head(10) # 상위 10개 도서 선택
top_books
# Groups:   book_code [10]
  user_code book_code score book_title
reviews avg_score
  <chr>      <chr>      <dbl> <chr>
<int>      <dbl>
1 US000274 BC0011134 5 세월호, 그날의 기록
4 5
2 US000950 BC0018957 5 웅크린 말들
1 5
3 US000183 BC0008295 5 매잡이
5 4.8
4 US000307 BC0012317 5 일방통행로 / 사유이미지
7 4.79
5 US000086 BC0006213 5 A Game of Thrones
4 4.75
6 US001084 BC0023719 5 소설 영웅문 제1부 - 몽고의 별
6 4.75
7 US000159 BC0007898 5 눈물도 빛을 만나면 반짝인다
9 4.72
8 US000130 BC0007430 4 여자는 인질이다
7 4.71
```

9	US001327	BC0027795	4.5	소설 영웅문 제2부 - 영웅의 별
5			4.7	
10	US000291	BC0001481	5	사자왕 형제의 모험
3			4.67	

평점이 낮지만 리뷰가 많은 Book 10개

```
# 평점이 낮고 리뷰가 많은 상위 10개 도서 찾기
bottom_books <- book_train %>%
  filter(avg_score < 3) %>% # 평균 평점이 낮은 도서 필터링 (예:
  # 평점이 3 미만인 책)
  arrange(desc(reviews)) %>% # 리뷰 수가 많은 순서로 정렬
  head(10) # 상위 10개 도서 선택
```



```
bottom_books
# Groups:   book_code [10]
  user_code book_code score book_title      rev
  <chr>      <chr>      <dbl> <chr>      <
1 US000001  BC0000013  4    아프니까 청춘이다  44
2          1.84
2 US000003  BC0000190  2.5  마시멜로 이야기  43
5          2.92
3 US000004  BC0000642  2    언어의 온도      3
24          2.54
4 US000008  BC0000030  0.5  멈추면, 비로소 보이는 것들  234
2.92
5 US000007  BC0001334  2    시크릿
232          2.15
6 US000004  BC0001027  4    하악하악          2
18          2.55
7 US000004  BC0000886  3.5  오체 불만족        2
07          2.58
8 US000003  BC0000252  4    지도 밖으로 행군하라  183
2.40
```

9	US000003	BC0000240	2	은교	
167		2.96			
10	US000007	BC0001349	0.5	꿈꾸는 다락방	16
5		2.48			

Task3

- user_code는 도서 데이터와 영화 데이터에서 공통으로 매겨진 것으로 같은 code는 동일 사용자를 의미한다. 도서와 영화 모두에 대해서 평점 기록이 있는 사용자들만을 추려서 아래의 내용을 분석해보자.
- 영화를 많이 보는(평점 기록이 많은) 사용자는 도서에 대한 평점 기록도 많은 경향이 있는가?
- 비교적 관대하게 평점을 매기는 사용자가 있는가 하면 평점이 박한 사용자가 있을 것이다. 사용자의 이러한 경향은 영화와 도서에 대해 동일하게 나타나는가 아니면 서로 다른가?
- i.e. 영화 평점이 generous한 사용자는 도서 평점도 generous한 경향이 있는가? 그 반대는 어떠한가?

확인해야 하는 것 (두 가지 가설)

1. 영화에 대한 평점 기록이 많은 사용자는 도서에 대한 평점 기록도 많은 경향이 있을 것이다. (영화 평점 개수 VS 도서 평점 개수)

Users who rate movies a lot tend to also rate books a lot.

2. 영화 평점이 generous한 사용자는 도서 평점도 generous한 경향이 있을 것이다. (영화 평균 평점 VS 도서 평균 평점)

Users who give generous ratings to movies tend to also give generous ratings to books.

진행할 실험

1. 상관계수 확인 (Check the correlation coefficient)
2. 산점도 그래프 (Create a scatter plot)
3. Quantile별 평균 값 확인 (Check the average values by quantiles)

1. 공통 평점이 있는 사용자만 필터링

```
user_code <- intersect(movie_df$user_code, book_df$user_code)

movie_df <- movie_df[movie_df$user_code %in% user_code, ]
book_df <- book_df[book_df$user_code %in% user_code, ]
```

영화를 많이 보는 사용자는 도서에 대한 평점기록도 많은가?

- 사용자에 대한 영화 리뷰 수, 도서 리뷰 수를 포함하는 df 만들기

```
movie_count <- movie_df %>%
  group_by(user_code) %>%
  summarise(movie_count = n())

book_count <- book_df %>%
  group_by(user_code) %>%
  summarise(book_count = n())

count_df <- merge(movie_count, book_count, on = "user_code")
```

▼ result

user_code	movie_count	book_count
US000001	189	42
US000002	824	7
US000003	1394	248
US000004	2684	457
US000005	639	12
US000006	765	93
US000007	533	250
US000008	448	66
US000009	751	250
US000010	853	308
US000011	468	55
US000012	2603	657
US000013	1239	32
US000014	1189	388

1. 상관계수 계산

- count_df의 movie_count와 book_count의 상관계수

```
### 상관계수 계산
mean_df2 <- mean_df
str(mean_df2)

# 상관계수 계산
```

```
correlation = cor(mean_df$movie_mean, mean_df$book_mean)
correlation
```

- 0.39
- 약한 양의 상관관계, 영화를 많이 본다고 해서 도서 평점 기록이 반드시 많은 것은 아니지만, 어느정도 경향성이 있음

2. 산점도 그래프 확인

```
#### 2. 산점도 그래프 그리기
str(count_df2)

# 이상치 처리
calculate_upper_threshold <- function(x) {
  Q1 <- quantile(x, 0.25)
  Q3 <- quantile(x, 0.75)
  IQR <- Q3 - Q1
  upper_threshold <- Q3 + 1.5 * IQR
  return(upper_threshold)
}

threshold_movie <- calculate_upper_threshold(count_df2$movie_count)
threshold_book <- calculate_upper_threshold(count_df2$book_count)

count_df2_filtered <- count_df2 %>%
  filter(movie_count <= threshold_movie & book_count <= threshold_book)

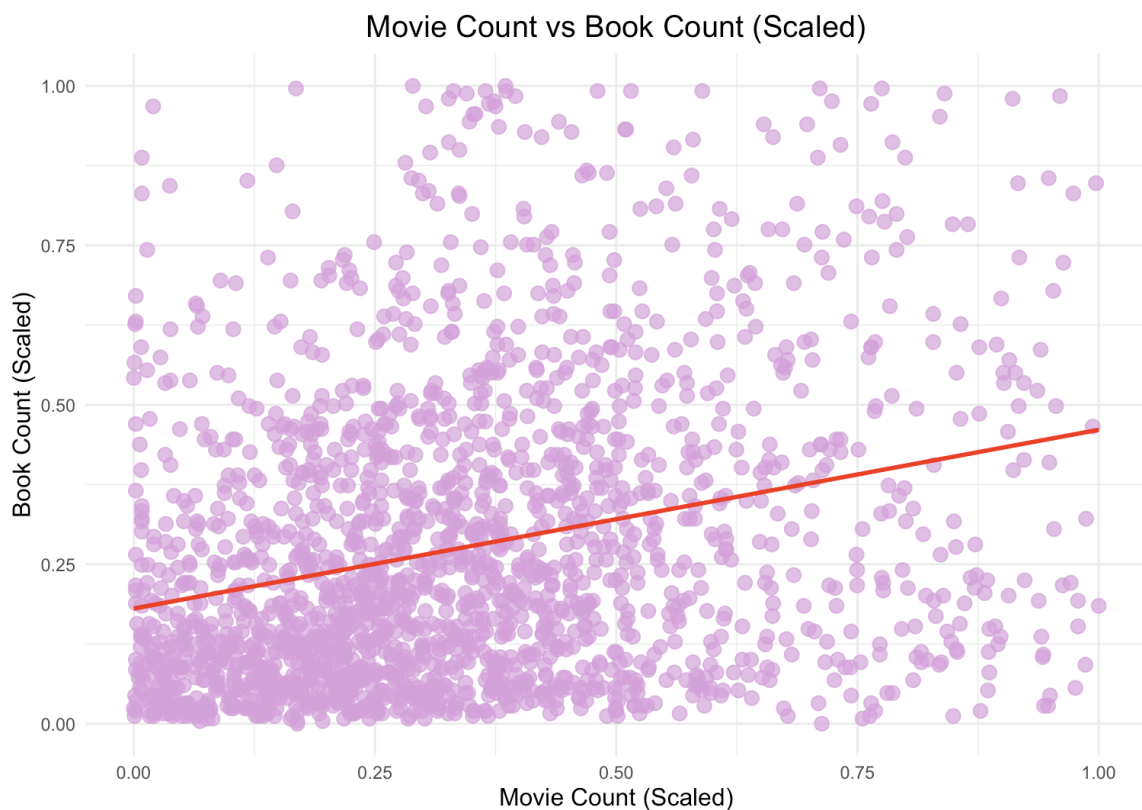
# 스케일링
count_df2_filtered <- count_df2_filtered %>%
  mutate(movie_count_scaled = rescale(movie_count),
         book_count_scaled = rescale(book_count))

# 그래프 그리기
ggplot(count_df2_filtered, aes(x = movie_count_scaled, y = book_count_scaled)) +
  geom_point(color = "plum", size = 3, alpha = 0.7) +
  labs(title = "Movie Count vs Book Count (Scaled)",
       x = "Movie Count (Scaled)",
       y = "Book Count (Scaled)")
```

```

    y = "Book Count (Scaled)") +
  theme_minimal() + # Clean theme
  theme(
    plot.title = element_text(hjust = 0.5, size = 15),
    title and adjust size
    axis.title.x = element_text(size = 12),
    axis.title.y = element_text(size = 12)
  ) +
  geom_smooth(method = "lm", color = "red", se = FALSE)

```



- 상관계수와 동일하게 약한 양의 상관관계를 보인다.

3. movie_count를 5개의 quantile로 나눈 후 quantile별 평균 도서 리뷰 수 확인

```

# movie_count를 5개의 분위로 나누기
count_df$movie_quantile <- cut(count_df$movie_count,
                                breaks = quantile(count_df$,
                                                    include.lowest = TRUE,

```



```
labels = FALSE)

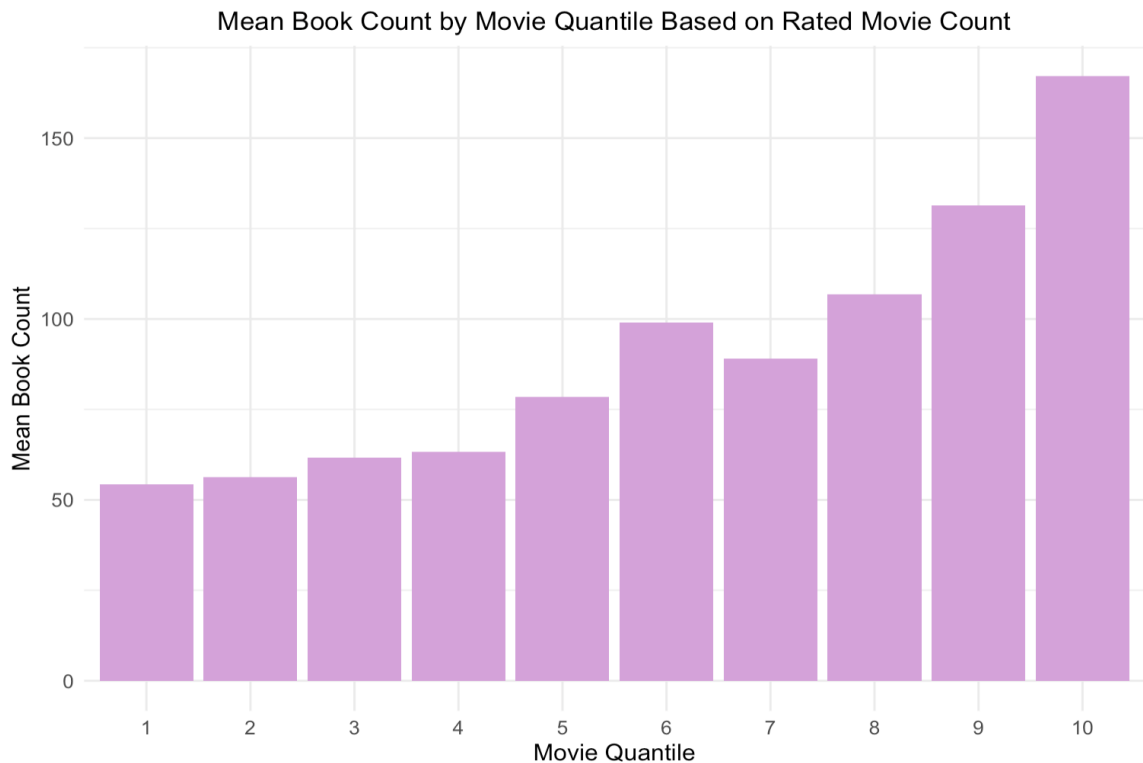
result <- count_df %>%
  group_by(movie_quantile) %>%
  summarise(mean_book_count = mean(book_count, na.rm = TRUE))
```

▼ result

movie_quantile	mean_book_mean
1 (6 ~ 121)	54.19369
2 (121 ~ 234)	56.25000
3 (234 ~ 320)	61.74429
4 (320 ~ 400)	63.29224
5 (400 ~ 485)	78.50679
6 (485 ~ 571)	99.04110
7 (571 ~ 685)	88.99543
8 (685 ~ 848)	106.78341
9 (848 ~ 1149)	131.35455
10 (1149 ~ 5446)	167.21101

4. 시각화

```
ggplot(result, aes(x = as.factor(movie_quantile), y = mean_book_count)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  labs(title = "Movie Quantile vs. Mean Book Count",
       x = "Movie Quantile",
       y = "Mean Book Count") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    axis.title.x = element_text(size = 12),
    axis.title.y = element_text(size = 12),
    axis.text.x = element_text(size = 10),
    axis.text.y = element_text(size = 10)
  )
```



영화 평점이 generous한 사용자는 도서 평점도 generous한가?

- 사용자에게 대한 영화 평균 평점, 도서 평균 평점을 포함하는 df 만들기

```
movie_mean <- movie_df %>%  
  group_by(user_code) %>%  
  summarise(movie_mean = mean(score))  
  
book_mean <- book_df %>%  
  group_by(user_code) %>%  
  summarise(book_mean = mean(score))  
  
mean_df <- merge(movie_mean, book_mean, on = "user_code")
```

1. 상관계수 계산

- count_df의 movie_count와 book_count의 상관계수

```

#### 상관계수 계산
mean_df2 <- mean_df
str(mean_df2)

# 상관계수 계산
correlation = cor(mean_df$movie_mean, mean_df$book_mean)
correlation

```

◦ 0.59

2. 산점도

```

str(mean_df2)

# 그래프 그리기
ggplot(mean_df2_filtered, aes(x = movie_mean_scaled, y = book_mean_scaled)) +
  geom_point(color = "purple", size = 3, alpha = 0.7) + #
  labs(title = "Movie Mean vs Book Mean (Scaled)",
        x = "Movie Mean (Scaled)",
        y = "Book Mean (Scaled)") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 15),
    axis.title.x = element_text(size = 12),
    axis.title.y = element_text(size = 12)
  ) +
  geom_smooth(method = "lm", color = "red", se = FALSE)

```



3. movie_mean을 5개의 quantile로 나눈 후 quantile별 평균 도서 점수 확인

```
mean_df$movie_quantile <- cut(mean_df$movie_mean,
                              breaks = quantile(mean_df$m
                              include.lowest = TRUE,
                              labels = FALSE)

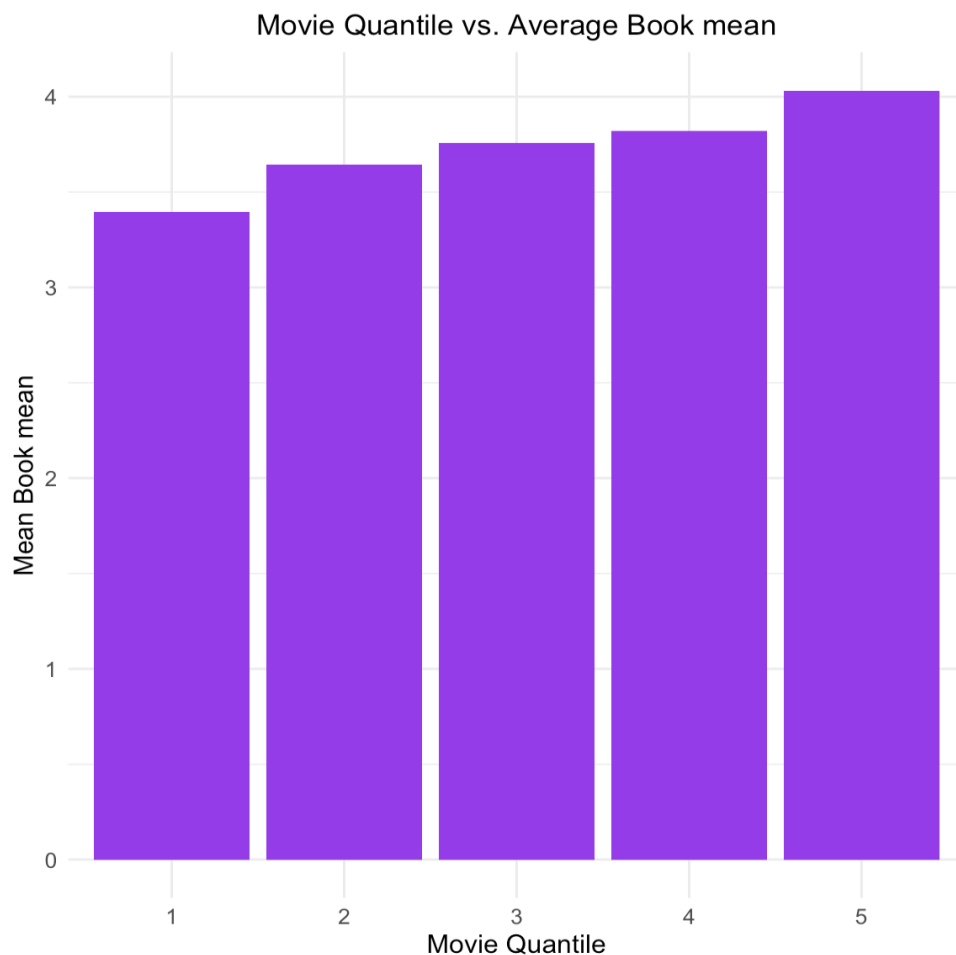
result <- mean_df %>%
  group_by(movie_quantile) %>%
  summarise(mean_book_mean = mean(book_mean, na.rm = TRUE))
```

▼ result

movie_quantile	mean_book_mean
1 (0.85 ~ 3.18)	3.40
2 (3.18 ~ 3.36)	3.65
3 (3.36 ~ 3.51)	3.76
4 (3.51 ~ 3.68)	3.82

5 (3.68 ~ 5.00)	4.03
-----------------	------

```
ggplot(result, aes(x = as.factor(movie_quantile), y = mean_book)) +
  geom_bar(stat = "identity", fill = "purple") +
  labs(title = "Movie Quantile vs. Average Book mean",
       x = "Movie Quantile",
       y = "Mean Book mean") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    axis.title.x = element_text(size = 12),
    axis.title.y = element_text(size = 12),
    axis.text.x = element_text(size = 10),
    axis.text.y = element_text(size = 10)
  )
```



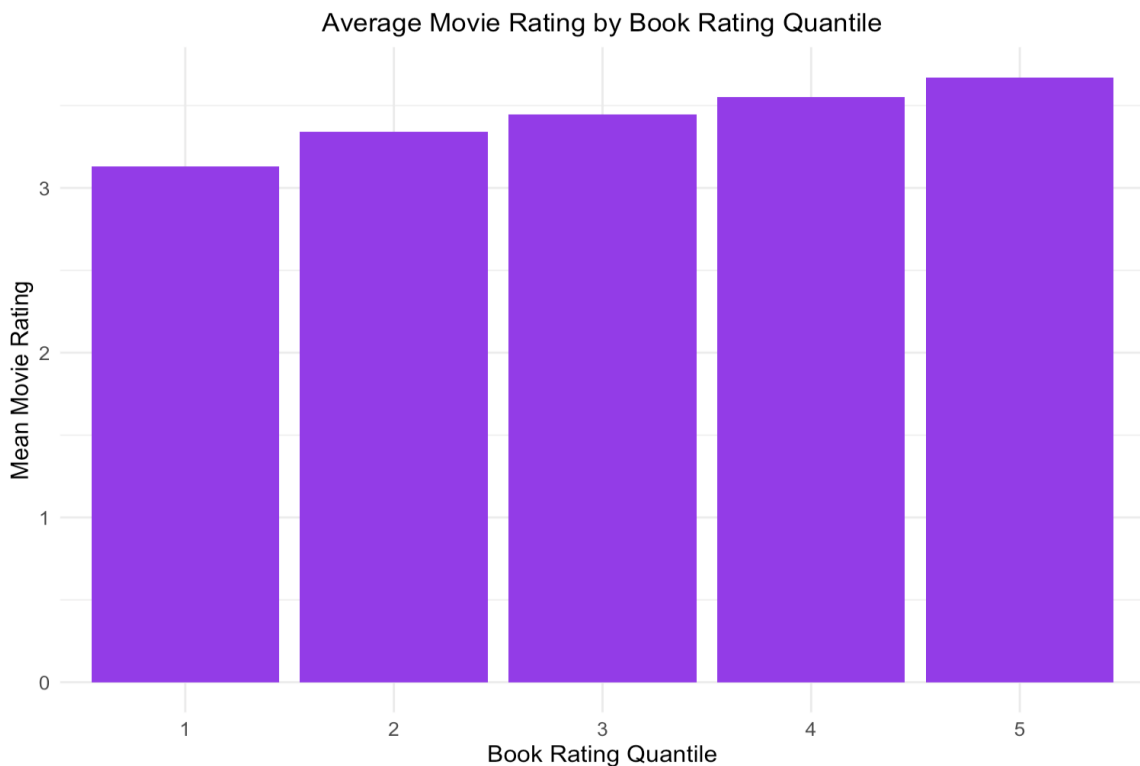
4. 도서 평점이 generous한 사용자는 영화 평점도 generous한가?

```
#### 3. book_mean을 5개의 분위로 나누기
mean_df$book_quantile <- cut(mean_df$book_mean,
                             breaks = quantile(mean_df$book_mean, n = 5),
                             include.lowest = TRUE,
                             labels = FALSE)

result <- mean_df %>%
  group_by(book_quantile) %>%
  summarise(mean_movie_mean = mean(movie_mean, na.rm = TRUE))

result

# 오 완전 양의 상관관계
# book_quantile book_count 평균 그래프 그리기
ggplot(result, aes(x = as.factor(book_quantile), y = mean_movie_mean)) +
  geom_bar(stat = "identity", fill = "purple") +
  labs(title = "Average Movie Rating by Book Rating Quantile",
       x = "Book Rating Quantile",
       y = "Mean Movie Rating") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    axis.title.x = element_text(size = 12),
    axis.title.y = element_text(size = 12),
    axis.text.x = element_text(size = 10),
    axis.text.y = element_text(size = 10)
  )
```



Task4: 영화 추천

UBCF와 IBCF를 통해 train 데이터를 사용해서 test 데이터의 평점을 예측

평점 예측의 MAE(Mean Absolute Error)를 계산하여 보고 얼마나 정확하게 평점을 예측하는지 확인

1. 데이터 형식 확인 및 변환

```
movie_train$user_code <- as.factor(movie_train$user_code)
movie_train$movie_code <- as.factor(movie_train$movie_code)
movie_train$score <- as.numeric(movie_train$score)
```

```
movie_test$user_code <- as.factor(movie_test$user_code)
movie_test$movie_code <- as.factor(movie_test$movie_code)
movie_test$score <- as.numeric(movie_test$score)
```

2. 데이터 넓은 형식으로 변환 (사용자-아이템 행렬)

```
train_matrix_wide <- dcast(movie_train, user_code ~ movie_code, value="score")
test_matrix_wide <- dcast(movie_test, user_code ~ movie_code, value="score")
head(train_matrix_wide)
```

```

# 3. 사용자 ID 열 제거 후 행렬로 변환
rownames(train_matrix_wide) <- train_matrix_wide$user_code
train_matrix <- as.matrix(train_matrix_wide[, -1])
rownames(test_matrix_wide) <- test_matrix_wide$user_code
test_matrix <- as.matrix(test_matrix_wide[, -1])

common_movies <- intersect(colnames(train_matrix), colnames(test_matrix))
train_matrix_filtered <- train_matrix[, common_movies]
test_matrix_filtered <- test_matrix[, common_movies]
# 2. 공통 영화만 포함한 테스트 데이터로 재구성
test_matrix_filtered <- test_matrix[, common_movies]

# 5. 행렬을 realRatingMatrix로 변환
train_realRatingMatrix <- as(train_matrix_filtered, "realRatingMatrix")
test_realRatingMatrix <- as(test_matrix_filtered, "realRatingMatrix")

# Build the User-Based Collaborative Filtering model
ubcf_model <- Recommender(train_realRatingMatrix, method = "UserBased")
ubcf_model
# Build the Item-Based Collaborative Filtering model
ibcf_model <- Recommender(train_realRatingMatrix, method = "ItemBased")
ibcf_model

# Predict ratings for the test data using UBCF
ubcf_predictions <- predict(ubcf_model, test_realRatingMatrix)

# Predict ratings for the test data using IBCF
ibcf_predictions <- predict(ibcf_model, test_realRatingMatrix)
ibcf_predictions
# MAE 계산 함수 (유효한 값이 있는지 점검)
calculate_mae <- function(predicted, actual) {
  pred_values <- as(predicted, "matrix")
  actual_values <- as(actual, "matrix")

  # 예측값과 실제값에서 NA가 아닌 값만 필터링
  valid_indices <- !is.na(pred_values) & !is.na(actual_values)

```



```

# 유효한 값이 있는 경우 MAE 계산, 없으면 경고 출력
if (sum(valid_indices) > 0) {
  mae <- mean(abs(pred_values[valid_indices] - actual_value
} else {
  mae <- NA
  cat("유효한 비교 대상이 없습니다. MAE를 계산할 수 없습니다.\n")
}

return(mae)
}

# Calculate MAE for UBCF
ubcf_mae <- calculate_mae(ubcf_predictions, test_realRatingMa

# Calculate MAE for IBCF
ibcf_mae <- calculate_mae(ibcf_predictions, test_realRatingMa

# Print MAE results
print(paste("UBCF MAE:", ubcf_mae))
print(paste("IBCF MAE:", ibcf_mae))

```

movie_train, movie_test 데이터의 사용자-유저 매트릭스를 각각 형성하고 train의 movie item과 test의 movie item이 동일해야 예측을 할 수 있다. 그래서 intersect 함수를 활용해서 공통 영화를 포함한 test 데이터를 재구성했습니다. 이를 각각 모델 학습을 하기 위해 'realRatingMatrix' 형태로 변환.

UBCF와 IBCF 모델을 만들고 예측했지만 MAE 값이 'NA' 값이 나와 계산하지 못했다.

```

> ubcf_mae <- calculate_mae(ubcf_predictions, test_realRatingMatrix)
유효한 비교 대상이 없습니다. MAE를 계산할 수 없습니다.
> # Calculate MAE for IBCF
> ibcf_mae <- calculate_mae(ibcf_predictions, test_realRatingMatrix)
유효한 비교 대상이 없습니다. MAE를 계산할 수 없습니다.
> # Print MAE results
> print(paste("UBCF MAE:", ubcf_mae))
[1] "UBCF MAE: NA"
> print(paste("IBCF MAE:", ibcf_mae))
[1] "IBCF MAE: NA"

```

```

# 예측된 평점(ubcf_predictions)의 NA 개수 출력
predicted_test_ratings <- as(ubcf_predictions, "matrix") # 여
actual_test_ratings <- as(test_realRatingMatrix, "matrix") #

# 예측된 평점에서 NA 개수 확인
pred_na_count <- sum(is.na(predicted_test_ratings))
cat("예측된 평점에서 NA 개수:", pred_na_count, "\n")

# 실제 평점에서 NA 개수 확인
actual_na_count <- sum(is.na(actual_test_ratings))
cat("실제 평점에서 NA 개수:", actual_na_count, "\n")

# 예측된 평점과 실제 평점에서 유효한 값(NA가 아닌 값) 확인
valid_count <- sum(!is.na(predicted_test_ratings) & !is.na(ac
cat("비교 가능한 데이터 수(NA가 아닌 값):", valid_count, "\n")

```

그 이유를 알아보기 위해 예측된 평점과 실제 평점의 NA값을 확인했고 예측한 평점과 실제 평점에서 유효한 값이 있는 지 확인했다.

그 결과, 각 NA개수가 매우 높았습니다. 예측된 평점과 실제 평점의 NA값이 너무 많아 추천을 못해줍니다.

```

> cat("예측된 평점에서 NA 개수:", pred_na_count, "\n")
예측된 평점에서 NA 개수: 23077159
> # 실제 평점에서 NA 개수 확인
> actual_na_count <- sum(is.na(actual_test_ratings))
> cat("실제 평점에서 NA 개수:", actual_na_count, "\n")
실제 평점에서 NA 개수: 24930438
> # 예측된 평점과 실제 평점에서 유효한 값(NA가 아닌 값) 확인
> valid_count <- sum(!is.na(predicted_test_ratings) & !is.na(actual_test_ratings))
> cat("비교 가능한 데이터 수(NA가 아닌 값):", valid_count, "\n")
비교 가능한 데이터 수(NA가 아닌 값): 0

```

Task5: 도서 추천

UBCF와 IBCF를 통해 train 데이터를 사용해서 test 데이터의 평점을 예측

평점 예측의 MAE(Mean Absolute Error)를 계산하여 보고 얼마나 정확하게 평점을 예측하는지 확인

UBCF

```
#예측하기
# 1. train 데이터와 test 데이터를 결합
combined_data <- rbind(book_train, book_test)

# 2. test 데이터의 평점을 NA로 설정
combined_data[combined_data$user_code %in% book_test$user_code, "score"] <- NA

# 3. 결합된 데이터를 realRatingMatrix로 변환
combined_matrix <- as(combined_data, "realRatingMatrix")

# 4. UBCF 모델 학습
ubcf_model <- Recommender(combined_matrix, method = "UBCF")

# 5. NA로 설정된 테스트 데이터의 평점을 예측
ubcf_prediction <- predict(ubcf_model, combined_matrix, type = "ratings")

# 6. 예측된 평점을 행렬 형식으로 변환
predicted_ratings <- as(ubcf_prediction, "matrix")

# 7. 테스트 데이터에 해당하는 예측된 평점만 추출
# book_test의 user_code와 book_code에 해당하는 예측 평점만 필터링
test_users <- book_test$user_code
test_books <- book_test$book_code
```

```

# 테스트 데이터의 유저와 책 코드에 해당하는 예측 평점을 정확히 매칭하여
추출
predicted_test_ratings <- matrix(NA, nrow = length(test_users), ncol = 1)
for (i in 1:length(test_users)) {
  user <- test_users[i]
  book <- test_books[i]
  predicted_test_ratings[i] <- predicted_ratings[user, book]
}

# 8. 실제 test 데이터에서 평점 추출
# 실제 평점도 test 데이터에 맞게 추출하여 크기 일치
actual_test_ratings <- book_test$score

# 9. MAE 계산 함수 정의
calculate_mae <- function(predicted, actual) {
  # 예측값(NA)이 아닌 값들만 비교
  valid_idx <- !is.na(predicted) & !is.na(actual)

  # 유효한 비교 값이 없을 경우 NA 반환
  if (sum(valid_idx) == 0) {
    return(NA)
  }

  # 실제 값과 예측된 값의 차이의 절대값을 계산한 후 평균
  mae <- mean(abs(predicted[valid_idx] - actual[valid_idx]))
  return(mae)
}

# 10. 예측된 평점과 실제 평점 간의 MAE 계산 (필터링된 데이터만 처리)
mae <- calculate_mae(predicted_test_ratings, actual_test_ratings)

# 11. 결과 출력
print(paste("MAE:", mae))

```

우선 train데이터와 test 데이터를 병합하였다, 동일한 아이템에 대하여 추천을 진행해야 하는데, train데이터만을 가지고 학습한 후, test데이터를 예측하고자 하면 그 열(아이템)의 수가 일치하지 않기 때문에 제대로 된 추천이 불가능하다.

때문에 train데이터와 test데이터를 병합하며, test데이터의 평점에 대한 값을 NA로 바꾼 후, 모델을 학습시키고. 그렇게 NA값을 예측하게 한다. 이후 실제 test의 예측값과 병합된 train데이터에서의 test부분 예측값을 비교하여 mae값을 계산하는 방법을 사용하였다.

그러나 실제 결과는

```
> print(paste("MAE:", mae))  
[1] "MAE: NA"
```

제대로 나오지 않았다. 이 원인을 파악하기 위해

```
> print(sum(is.na(predicted_test_ratings))) # 예측된 평점에서  
NA 개수  
[1] 42454  
> print(sum(is.na(actual_test_ratings))) # 실제 평점에서 NA  
개수  
[1] 0  
> # 예측된 평점과 실제 평점에서 유효한 값 확인  
> print(sum(!is.na(predicted_test_ratings) & !is.na(actual_  
test_ratings))) # 비교 가능한 데이터 수  
[1] 0
```

다음과 같이 NA개수를 세었는데, 병합한 후 매트릭스에서 예측된 평점에서 NA값이 너무 많아 비교가 불가능하였다.

```
unique(book_train$book_code)
```

실제 매트릭스로 변환을 한다면

```
[ reached getOption("max.print") -- omitted 4865 entries ]
```

4865개의 unique한 book_code가 전부 열로 들어갈 것이고, 이것이 또 각 user별로 NA 값으로 들어가서 실제로는 1000만가지 이상의 결측치가 발생해, 예측 자체가 안될 것이다.

IBCF

```
# IBCF
# 1. train 데이터와 test 데이터를 결합
combined_data <- rbind(book_train, book_test)

# 2. test 데이터의 평점을 NA로 설정
combined_data[combined_data$user_code %in% book_test$user_code, "score"] <- NA

# 3. 결합된 데이터를 realRatingMatrix로 변환
combined_matrix <- as(combined_data, "realRatingMatrix")

# 4. IBCF 모델 학습
ibcf_model <- Recommender(combined_matrix, method = "IBCF")

# 5. NA로 설정된 테스트 데이터의 평점을 예측
ibcf_prediction <- predict(ibcf_model, combined_matrix, type = "ratings")

# 6. 예측된 평점을 행렬 형식으로 변환
predicted_ratings <- as(ibcf_prediction, "matrix")

# 7. 테스트 데이터에 해당하는 예측된 평점만 추출
test_users <- book_test$user_code
test_books <- book_test$book_code

# 테스트 데이터의 유저와 책 코드에 해당하는 예측 평점을 정확히 매칭하여
```

추출

```
predicted_test_ratings <- matrix(NA, nrow = length(test_users), ncol = 1)
for (i in 1:length(test_users)) {
  user <- test_users[i]
  book <- test_books[i]
  predicted_test_ratings[i] <- predicted_ratings[user, book]
}
```

8. 실제 test 데이터에서 평점 추출

```
actual_test_ratings <- book_test$score
```

9. MAE 계산 함수 정의

```
calculate_mae <- function(predicted, actual) {
  # 결측값(NA)이 아닌 값들만 비교
  valid_idx <- !is.na(predicted) & !is.na(actual)

  # 유효한 비교 값이 없을 경우 NA 반환
  if (sum(valid_idx) == 0) {
    return(NA)
  }

  # 실제 값과 예측된 값의 차이의 절대값을 계산한 후 평균
  mae <- mean(abs(predicted[valid_idx] - actual[valid_idx]))
  return(mae)
}
```

10. 예측된 평점과 실제 평점 간의 MAE 계산

```
mae <- calculate_mae(predicted_test_ratings, actual_test_ratings)
```

11. 결과 출력

```
print(paste("MAE:", mae))
```

동일하게, MAE의 값이 NA로 나왔다.

특정 유저에 대해서 예측하기

```
## 다시 마지막 US000003 예측
# 1. train 데이터와 test 데이터를 병합
combined_data <- rbind(book_train, book_test)

# 2. US000003의 book_code 추출 (train + test 데이터에서 평가한
# 모든 책)
us000003_books <- combined_data %>%
  filter(user_code == "US000003") %>%
  select(book_code) %>%
  distinct()

# 3. US000003과 가장 많이 공통 책을 읽은 상위 30명의 사용자 찾기
similar_users <- combined_data %>%
  filter(book_code %in% us000003_books$book_code) %>%
  group_by(user_code) %>%
  summarise(common_books = n_distinct(book_code)) %>%
  arrange(desc(common_books)) %>%
  filter(user_code != "US000003") %>%
  head(30) %>%
  select(user_code)

# 4. similar_users와 US000003의 데이터를 필터링
similar_users_data <- combined_data %>%
  filter(user_code %in% similar_users$user_code | user_code
== "US000003")

# 5. US000003의 test 데이터 점수를 NA로 변환
us000003_test_na <- book_test %>%
  filter(user_code == "US000003") %>%
  mutate(score = NA)

# 6. combined_data에 test 데이터에서 NA 처리된 US000003의 데이터
# 추가
combined_data_na <- rbind(similar_users_data, us000003_test
_na)
```



```

# 7. realRatingMatrix로 변환
rating_matrix_real <- as(combined_data_na, "realRatingMatrix")

# 8. realRatingMatrix를 매트릭스로 변환
rating_matrix <- as(rating_matrix_real, "matrix")

# 9. NA 비율이 특정 비율 이상인 열(책) 삭제 (예: NA가 50% 이상인 열 삭제)
na_threshold <- 0.5 # 50% 이상 NA가 있는 열을 삭제하기 위한 기준
cols_to_keep <- colSums(is.na(rating_matrix)) / nrow(rating_matrix) < na_threshold
filtered_rating_matrix <- rating_matrix[, cols_to_keep]

# 10. 결과 확인 (필터링된 매트릭스)
View(filtered_rating_matrix)

# 1. 필터링된 매트릭스를 realRatingMatrix로 변환
filtered_rating_matrix_real <- as(filtered_rating_matrix, "realRatingMatrix")

# 2. UBCF 모델 학습
ubcf_model_filtered <- Recommender(filtered_rating_matrix_real, method = "UBCF")

# 3. US000003에 대한 평점 예측
predicted_ratings_us000003_filtered <- predict(ubcf_model_filtered, filtered_rating_matrix_real["US000003", ], type = "ratings")

# 4. 예측된 평점을 매트릭스로 변환
predicted_ratings_us000003_filtered_matrix <- as(predicted_ratings_us000003_filtered, "matrix")

# 5. 결과 확인 (US000003에 대한 예측된 평점)
View(predicted_ratings_us000003_filtered_matrix)

```

```

# 1. US000003에 대한 예측된 평점 추출 (이미 수행된 상태)
predicted_test_books_filtered <- predicted_ratings_us000003
_filtered_matrix

# 2. book_test에서 US000003의 실제 평점 데이터 추출
actual_ratings_us000003 <- book_test %>%
  filter(user_code == "US000003") %>%
  select(book_code, score)

# 3. 예측된 책과 실제 책의 book_code를 비교하여 공통된 book_code 필터링
common_books <- intersect(colnames(predicted_test_books_filtered),
  actual_ratings_us000003$book_code)

# 4. 공통된 book_code에 대해 예측된 평점과 실제 평점 필터링
predicted_test_books_filtered_common <- predicted_test_books_filtered[,
  common_books, drop = FALSE]
actual_ratings_filtered_common <- actual_ratings_us000003 %>%
  filter(book_code %in% common_books)

# 5. 예측된 값과 실제 값 비교 데이터프레임 생성
comparison <- data.frame(
  book_code = common_books,
  predicted_score = as.vector(predicted_test_books_filtered_common),
  actual_score = actual_ratings_filtered_common$score
)

# 6. 결과 출력
print(comparison)
actual_ratings_filtered_common <- actual_ratings_us000003 %>%
  filter(book_code %in% common_books)

# 5. 예측된 값과 실제 값 비교 데이터프레임 생성
comparison <- data.frame(
  book_code = common_books,

```

```

predicted_score = as.vector(predicted_test_books_filtered
_common),
actual_score = actual_ratings_filtered_common$score
)

# 6. 결과 출력
print(comparison)

```

개념:

현재 예측이 불가능한 이유는 train데이터를 매트릭스로 변환하는 과정에서 NA값이 매우 많이 발생하며, 이 이유는 먼저 user별로 읽은 book이 다르고, 그 중에서도 서로 교집합이 매우 부족하며, 이를 매트릭스로 변환하는 과정에서 생기는 book_code의 열이 굉장히 많이 발생하기 때문이다.

이를 해결하기 위해서, 우선 한 명의 특정한 user를 찾아서, 그 user와 읽은 book의 종류가 비슷한 상위 30명을 찾고, 그 사람들에게 대해서 매트릭스를 만든 후, 그 안에서 다시 각 열별로 NA값의 비율이 50%가 넘어가는 열들을 제거하여 예측을 준비하였다.

이후 우리가 원하는 test데이터에 대해서는 해당 user의 값을 NA처리한 후, train데이터와 test데이터를 병합한 후 예측을 수행하였고, 그 결과 예측이 어느정도 성공하였다.

```

> print(comparison)
  book_code predicted_score actual_score
1  BC0000051          4.509424          3.0
2  BC0000094          3.499434          4.0
3  BC0000096          3.466483          3.0
4  BC0000116          3.969298          4.0
5  BC0000120          3.713496          4.5
6  BC0000177          3.364017          2.5
7  BC0000232          3.839071          3.0
8  BC0000246          3.509058          4.0
9  BC0000280          4.288513          4.0
10 BC0000323          2.732721          4.0
11 BC0000341          3.707695          4.5

```

user_code US000003에 대한 예측이며, 실제 test데이터는 50가지였으나, 많은 사람들이 us000003의 test데이터에 있는 book을 읽지 않아 예측이 어려웠고, 그 때문에 실제 예측이 가능한 book은 11가지였다.

```
# MAE 계산 함수 정의
calculate_mae <- function(predicted, actual) {
  # NA가 아닌 값들만 비교
  valid_idx <- !is.na(predicted) & !is.na(actual) # NA가 아닌 유효한 값들만 필터링
  mae <- mean(abs(predicted[valid_idx] - actual[valid_idx])) # 절대값 차이의 평균 계산
  return(mae)
}

# comparison 데이터프레임에서 예측값과 실제값을 추출하여 MAE 계산
mae <- calculate_mae(comparison$predicted_score, comparison$actual_score)
```

그래서 예측된 값들에 대해 mae를 계산해 본 결과

```
> print(paste("Mean Absolute Error (MAE):", mae))
[1] "Mean Absolute Error (MAE): 0.712346"
```

0.712346이 나온다.

이제 이 코드를 가지고 다른 사람에 대해서도 수행을 해보고자 한다.

```
# 1. train 데이터를 realRatingMatrix로 변환
train_matrix <- as(book_train, "realRatingMatrix")

# 2. UBCF 모델 학습 (train 데이터만 사용)
total_users_train <- nrow(train_matrix) # train 데이터의 전체
```

사용자 수

```
k_value_train <- round(total_users_train * 0.1) # train 데이터의 사용자 수의 10%를 K값으로 설정
```

```
ubcf_model_train <- Recommender(train_matrix, method = "UBCF", parameter = list(nn = k_value_train))
```

3. train 데이터에 대한 예측 수행 (train 데이터로 학습된 모델로 train 데이터에 대해 예측)

```
ubcf_prediction_train <- predict(ubcf_model_train, train_matrix, type = "ratings")
```

4. 예측된 평점을 행렬 형식으로 변환 (train 데이터에서 예측된 결과)

```
predicted_ratings_train <- as(ubcf_prediction_train, "matrix")
```

```
View(predicted_ratings_train)
```

5. 테스트 데이터에 해당하는 예측된 평점만 추출

book_test의 user_code와 book_code에 해당하는 예측 평점만 필터링

```
test_users <- book_test$user_code
```

```
test_books <- book_test$book_code
```

테스트 데이터의 유저와 책 코드에 해당하는 예측 평점을 정확히 매칭하여 추출

```
predicted_test_ratings <- matrix(NA, nrow = length(test_users), ncol = 1)
```

```
for (i in 1:length(test_users)) {
```

```
  user <- test_users[i]
```

```
  book <- test_books[i]
```

train에서 예측된 값 중, test 데이터와 동일한 user와 book에 대한 예측값 추출

```
  if (!is.na(predicted_ratings_train[user, book])) {  
    predicted_test_ratings[i] <- predicted_ratings_train[user, book]  
  }  
}
```

```
}
```

```
View(predicted_test_ratings)
```

6. 실제 test 데이터에서 평점 추출

```
actual_test_ratings <- book_test$score
```

```

# 7. MAE 계산 함수 정의 (이전과 동일)
calculate_mae <- function(predicted, actual) {
  # 결측값(NA)이 아닌 값들만 비교
  valid_idx <- !is.na(predicted) & !is.na(actual)

  # 유효한 비교 값이 없을 경우 NA 반환
  if (sum(valid_idx) == 0) {
    return(NA)
  }

  # 실제 값과 예측된 값의 차이의 절대값을 계산한 후 평균
  mae <- mean(abs(predicted[valid_idx] - actual[valid_idx]))
  return(mae)
}

# 8. 예측된 평점과 실제 평점 간의 MAE 계산 (필터링된 데이터만 처리)
mae <- calculate_mae(predicted_test_ratings, actual_test_ratings)

# 9. 결과 출력
print(paste("MAE:", mae))

```

이 데이터 자체는 도서에 대한 전체 데이터이며, 당연하지만 한 사람이 읽는 도서는 전체 도서의 양의 1%도 채 되지 않을 것이다. 당연히 매트릭스에서 NA값이 많이 나올 것이고, 그 안에서 우리는 추천을 하는 것을 목표로 하기 때문에 데이터를 추가적으로 처리하는 것은 큰 의미가 없다.

전체 데이터에서 NA값을 처리하지 않고 예측하기

이제 NA값에 대한 처리를 하지 않은 상태에서 UBCF에 대해 예측하고, mae값을 구해야 한다. 결국 UBCF는 비슷한 유저를 찾고, 예측을 수행하기 때문에 이 K값이 굉장히 중요하며, 나는 이 k값을 상수로 활용하기보다는 전체 유저의 일정 비율로 정하고 예측하는 것이 필요하다고 생각했다.

```

# 1. train 데이터를 realRatingMatrix로 변환
train_matrix <- as(book_train, "realRatingMatrix")

# 2. UBCF 모델 학습 (train 데이터만 사용)
total_users_train <- nrow(train_matrix) # train 데이터의 전체
사용자 수
k_value_train <- round(total_users_train * 0.1) # train 데
이터의 사용자 수의 10%를 K값으로 설정

ubcf_model_train <- Recommender(train_matrix, method = "UBC
F", parameter = list(nn = k_value_train))

# 3. train 데이터에 대한 예측 수행 (train 데이터로 학습된 모델로 tra
in 데이터에 대해 예측)
ubcf_prediction_train <- predict(ubcf_model_train, train_ma
trix, type = "ratings")

# 4. 예측된 평점을 행렬 형식으로 변환 (train 데이터에서 예측된 결과)
predicted_ratings_train <- as(ubcf_prediction_train, "matri
x")
View(predicted_ratings_train)
# 5. 테스트 데이터에 해당하는 예측된 평점만 추출
# book_test의 user_code와 book_code에 해당하는 예측 평점만 필터링
test_users <- book_test$user_code
test_books <- book_test$book_code

# 테스트 데이터의 유저와 책 코드에 해당하는 예측 평점을 정확히 매칭하여
추출
predicted_test_ratings <- matrix(NA, nrow = length(test_use
rs), ncol = 1)
for (i in 1:length(test_users)) {
  user <- test_users[i]
  book <- test_books[i]
  # train에서 예측된 값 중, test 데이터와 동일한 user와 book에 대한
예측값 추출
  if (!is.na(predicted_ratings_train[user, book])) {
    predicted_test_ratings[i] <- predicted_ratings_train[us
er, book]
  }
}

```

```

    }
  }
  View(predicted_test_ratings)
  # 6. 실제 test 데이터에서 평점 추출
  actual_test_ratings <- book_test$score

  # 7. MAE 계산 함수 정의 (이전과 동일)
  calculate_mae <- function(predicted, actual) {
    # 예측값(NA)이 아닌 값들만 비교
    valid_idx <- !is.na(predicted) & !is.na(actual)

    # 유효한 비교 값이 없을 경우 NA 반환
    if (sum(valid_idx) == 0) {
      return(NA)
    }

    # 실제 값과 예측된 값의 차이의 절대값을 계산한 후 평균
    mae <- mean(abs(predicted[valid_idx] - actual[valid_idx]))
    return(mae)
  }

  # 8. 예측된 평점과 실제 평점 간의 MAE 계산 (필터링된 데이터만 처리)
  mae <- calculate_mae(predicted_test_ratings, actual_test_ratings)

  # 9. 결과 출력
  print(paste("MAE:", mae))

```

```

> print(paste("MAE:", mae))
[1] "MAE: 0.650872137070215"

```

실제로 K값이 전체 매트릭스의 행의 10%와 같은 비율로 나타나자, 예측이 성공하였고, 그 값이 확인이 되었다.

다음으로는 IBCF에 대해서도 예측을 시도하였다, train데이터에 대해서 예측을 시도한 후, 예측된 값들 중에서 실제 값과 일치하는 항목만을 계산해 mae값을 계산하였다.

```
# 1. train 데이터를 realRatingMatrix로 변환
train_matrix <- as(book_train, "realRatingMatrix")

# 2. IBCF 모델 학습 (train 데이터만 사용)
ibcf_model_train <- Recommender(train_matrix, method = "IBCF")

# 3. train 데이터에 대한 예측 수행 (train 데이터로 학습된 모델로 train 데이터에 대해 예측)
ibcf_prediction_train <- predict(ibcf_model_train, train_matrix, type = "ratings")

# 4. 예측된 평점을 행렬 형식으로 변환 (train 데이터에서 예측된 결과)
predicted_ratings_train_ibcf <- as(ibcf_prediction_train, "matrix")

# 5. 테스트 데이터에 해당하는 예측된 평점만 추출
# book_test의 user_code와 book_code에 해당하는 예측 평점만 필터링
test_users <- book_test$user_code
test_books <- book_test$book_code

# 테스트 데이터의 유저와 책 코드에 해당하는 예측 평점을 정확히 매칭하여 추출
predicted_test_ratings_ibcf <- matrix(NA, nrow = length(test_users), ncol = 1)
for (i in 1:length(test_users)) {
  user <- test_users[i]
  book <- test_books[i]
  # train에서 예측된 값 중, test 데이터와 동일한 user와 book에 대한 예측값 추출
  if (!is.na(predicted_ratings_train_ibcf[user, book])) {
    predicted_test_ratings_ibcf[i] <- predicted_ratings_train_ibcf[user, book]
  }
}
```

```

# 6. 실제 test 데이터에서 평점 추출
actual_test_ratings <- book_test$score

# 7. MAE 계산 함수 정의 (이전과 동일)
calculate_mae <- function(predicted, actual) {
  # 예측값(NA)이 아닌 값들만 비교
  valid_idx <- !is.na(predicted) & !is.na(actual)

  # 유효한 비교 값이 없을 경우 NA 반환
  if (sum(valid_idx) == 0) {
    return(NA)
  }

  # 실제 값과 예측된 값의 차이의 절대값을 계산한 후 평균
  mae <- mean(abs(predicted[valid_idx] - actual[valid_idx]))
  return(mae)
}

# 8. 예측된 평점과 실제 평점 간의 MAE 계산 (IBCF 모델 결과에 대해 처리)
mae_ibcf <- calculate_mae(predicted_test_ratings_ibcf, actual_test_ratings)

# 9. 결과 출력
print(paste("MAE (IBCF):", mae_ibcf))

```

```

> print(paste("MAE (IBCF):", mae_ibcf))
[1] "MAE (IBCF): 0.741644414454879"

```

Task6

- 자신이 전에 감상했던 영화 20편에 대해서 본인의 평가를 담은 평점을 기록하고, 자신에게 추천을 수행해보라. 어떤 영화가 자신에게 추천되었으며 reasonable 추천 결과인지 설명해보라.

- 동일하게 도서에 대해서도 20개의 평점을 데이터에 추가한 후 자신에 대한 도서 추천을 수행하고, 결과에 대해서 평가해보라.

▼ 영화 20편에 대한 평가 (my_movie)

영화 코드	영화이름	평점
MC0000024	미 비포 유	3.0
MC0000400	주토피아	4.5
MC0000052	미니언즈	3.5
MC0000053	뷰티 인사이드	3.0
MC0000055	박물관이 살아있다!	4.5
MC0000065	센과 치히로의 행방불명	4.5
MC0000077	유령 신부	5
MC0000089	마션	2.0
MC0000100	인사이드 아웃	4.5
MC0000110	어벤져스	4.5
MC0000124	아바타	3.5
MC0000134	찰리와 초콜릿 공장	4.0
MC0000136	7번방의 선물	4.0
MC0000139	스쿨 오브 락	4.5
MC0000171	해운대	3.5
MC0000175	세 열간이	5
MC0000182	님아, 그 강을 건너지 마오	4.0
MC0000306	리틀 포레스트	4.5
MC0000319	코코	4.5
MC0000326	진격의 거인-자유의 날개-	4.5
MC0000355	모아나	4.0
MC0000416	암살	4.0
MC0000418	컨저링	4.0
MC0000676	헝거게임: 판엠의 불꽃	4.5
MC0000865	라퐁젤	4.5
MC0000900	늑대소년	4.0

MC0000929	키싱 부스	2.0
MC0001715	코렐라인: 비밀의 문	5

▼ 도서 20권에 대한 평가 (my_book)

book	도서이름	평점
BC0000043	정의란 무엇인가	4.0
BC0000050	위대한 개츠비	3.0
BC0000091	꽃들에게 희망을	3.5
BC0000110	조명가게	4.5
BC0000155	무빙	4.5
BC0000190	마시멜로 이야기	5
BC0000201	우리들의 일그러진 영웅	3.0
BC0000235	노인과 바다	4.0
BC0000315	총 균 쇄	3.5
BC0000797	미움받을 용기	3.0
BC0004581	기억 전달자	4.5
BC0005676	파페포포 레인보우	5
BC0001062	오베라는 남자	4.5
BC0001338	멋진 신세계	3.5
BC0001344	인간 실격	3.5
BC0001642	후레자식	3.5
BC0001301	식객	4.5
BC0001729	자살가게	4.5
BC0005750	생명이 있는 것은 다 아름답다	4.5
BC0007101	프로테스탄트 윤리와 자본주의 정신	4.0
BC0000761	아파트	4.0
BC0001473	어게인	5
BC0002078	미움받을 용기 2	4.5

1) 영화에 대한 평가

1. 전처리

- train, test data 병합

```
# 더 정확한 결과를 위해 train, test 데이터 병합
movie_df <- rbind(movie_train, movie_test)
```

- 영화 코드와 이름을 매칭하기 위한 df 생성

```
movie_names <- movie_df[,c(2,4)]
movie_names <- distinct(movie_names)

# 영화 이름 제거
movie_df <- movie_df[, -4]
```

- 나의 영화 rating 가져오기

```
my_movie <- read.csv("my_movie.csv")

# 열 이름 통합
colnames(my_movie) <- c("user_code", "movie_code", "score")

# 임의의 user_code 배정
my_movie[,1] <- "US0000000"
```

2. 추천 시스템 모델링

- 나의 평가와 기존 평가 병합

```
ratings.df <- rbind(my_movie, movie_df)
colnames(ratings.df) <- c('user', 'movie', 'rating')
```

- spread 형식으로 변경

```
rating_matrix <- ratings.df %>%
  spread(movie, rating) %>%
```

```
remove_rownames() %>%
column_to_rownames(var = 'user')
```

- sparse matrix 형식으로 변경

```
rating_rrm <- as(as(rating_matrix, 'matrix'), 'realRatir
```

- 평가영화가 20개 이하인 user, 50개 이하의 평가가 있는 movie 제거

```
rating_rrm <- rating_rrm[rowCounts(rating_rrm) > 20,
                        colCounts(rating_rrm) > 50]
```

- 모델 만들기

```
# UBCF
ubcf_model <- Recommender(rating_rrm, method = 'UBCF')

# IBCF
ibcf_model <- Recommender(rating_rrm, method = 'IBCF')
```

- 예측

```
recom <- predict(ubcf_model, rating_rrm[1, ])
rownames(rating_rrm)[1]

> rownames(rating_rrm)[1]
[1] "US0000000"
```

3. 결과 확인

- 추천 영화 code 저장

```
recom_movie_code <- as(recom, 'list')
recom_movie_code <- unlist(recom_movie_code)
```

- 추천 영화 코드, 이름 병합

```
recom_movie <- movie_names[movie_names$movie_code %in% r
```

1. UBCF

movie_code	movie_title
MC0000702	천공의 성 라퓨타
MC0000875	바람계곡의 나우시카
MC0000988	클라우스
MC0001031	더 페이버릿: 여왕의 여자
MC0001373	빌리 엘리어트
MC0000103	신세계
MC0001828	판의 미로 - 오피리아와 세 개의 열쇠
MC0002595	파이퍼
MC0003056	12명의 성난 사람들
MC0006244	극장판 포켓몬스터 : 뮤츠의 역습

2. IBCF

movie_code	movie_title
MC0000013	타이타닉
MC0000064	어벤져스: 에이지 오브 울트론
MC0000114	토이 스토리 3
MC0000116	인셉션
MC0000133	아이언맨
MC0000153	하울의 움직이는 성
MC0000181	캡틴 아메리카: 윈터 솔져
MC0000038	캡틴 아메리카: 시빌 워
MC0000858	해리포터와 아즈카반의 죄수
MC0000123	아이언맨 3

- IBCF를 통해 얻은 예측 결과가 확실히 직관적이긴 하지만, 추천된 영화들은 모델을 사용하지 않아도 충분히 예상할 수 있는 작품들이다. 반면, UBCF의 결과는 조금 더 흥미로웠다.
- UBCF 결과로 추천된 영화들의 예고편을 확인해보니, 보라색으로 표시된 영화들은 개인적으로 흥미를 끌었다. 그러나 '신세계'나 '12명의 성난 사람들'과 같은 영화들은 원래 개인적으로 선호하지 않는 장르라서, 추천 시스템의 성능이 아주 뛰어나다고 보기는 어려울 것 같다.

2) 도서에 대한 평가

```
##### Book #####
# 좀 더 정확한 추천을 위해 train, test 합치기
book_df <- rbind(book_train, book_test)
```

```

# 책 이름 새로운 데이터 프레임으로 저장하기
book_names <- book_df[,c(2,4)]
book_names <- distinct(book_names)

# 이전 데이터프레임에서 책 이름 제거
book_df <- book_df[, -4]

# 나의 book rating 불러오기
my_book <- read.csv("my_book.csv")

colnames(my_book) <- c("user_code", "book_code", "score")

### 추천 알고리즘
# 나의 rating과 book_df 정보 합치기
ratings.df <- rbind(my_book, book_df)
colnames(ratings.df) <- c('user', 'book', 'rating')

# spread 형식으로 만들기
rating_matrix <- ratings.df %>%
  spread(book, rating) %>%
  remove_rownames() %>%
  column_to_rownames(var = 'user')

# sparse matrix
rating_rrm <- as(as(rating_matrix, 'matrix'), 'realRatingMatr

# 평점을 내린 영화가 20개 이상인 user, 영화 평점이 50개 이상인 book 필터
rating_rrm <- rating_rrm[rowCounts(rating_rrm) > 20,
                        colCounts(rating_rrm) > 50]

# 모델 만들기
ubcf_model <- Recommender(rating_rrm, method = 'UBCF')
ibcf_model <- Recommender(rating_rrm, method = 'IBCF')

# 예측
recom <- predict(ibcf_model, rating_rrm[1, ])
rownames(rating_rrm)[1]

```



```

recom_book_code <- as(recom, 'list')
recom_book_code <- unlist(recom_book_code)

# 영화 이름 확인
recom_book <- book_names[book_names$book_code %in% recom_book_code,]
recom_book

```

1. UBCF

movie_code	movie_title
BC0000191	오리엔트 특급 살인
BC0000751	이상한 나라의 앨리스
BC0000913	사람은 무엇으로 사는가
BC0001066	남자들은 자꾸 나를 가르치려 든다
BC0001100	개미
BC0001309	안나 카레니나
BC0001495	반 고흐, 영혼의 편지
BC0002145	하이큐!!
BC0002490	건지 감자껍질파 이 북클럽
BC0002590	아픔이 길이 되려면

2. IBCF

movie_code	movie_title
BC0000100	자기만의 방
BC0000274	꿈꾸는 책들의 도시
BC0000337	7년의 밤
BC0000471	여자 둘이 살고 있습니다
BC0000745	왜 세계의 절반은 굶주리는가?
BC0001096	파도가 바다의 일이라면
BC0001106	존재의 세 가지 거짓말
BC0001321	책은 도끼다
BC0000116	죽음에 관하여
BC0000278	소피의 세계

- 책을 많이 읽는 편이 아니어서 내가 준 평점의 책 종류가 다양하지 않아 결과가 큰 의미가 없을 것 같다고 생각했지만, 예상외로 내가 좋아하는 장르들이 적절하게 추천되었다.
- 시리즈물이 많고 장르가 명확한 영화와는 달리, 책은 속성이 비슷해도 새로운 책들이 많아서 이번에는 오히려 IBCF로 추천된 결과가 더 흥미롭게 느껴졌다.